

```
$ onboard jordanvalnet/code-dev-intel.ts --profile dev
```

code-dev-intel.ts remplace grep pour les agents IA : un serveur MCP TypeScript bien outillé, testé sur trois OS, porté par une seule personne

Self-hosted AI code intelligence for TypeScript — symbol resolution, structural search, duplicate detection, and dependency graphs via MCP server.

0

ÉTOILES GITHUB

1

CONTRIBUTEURS ACTIFS
sur 12 semaines

0

PR OUVERTES

6 sept. 2026

DERNIER COMMIT

Que fait ce projet et pour qui ?

> Le pitch : l'agent arrête de grep et navigue dans TypeScript avec le type-checker, en local, sous AGPL.

Un paquet npm qui expose un serveur MCP et HTTP auto-hébergé d'intelligence de code TypeScript : définitions, références, implémentations, plan d'un fichier, graphe d'imports, recherche structurelle, recherche texte et détection de doublons, sans que le modèle relise tout le repo.

Pour qui : les agents IA et assistants d'IDE (Claude Code, VS Code et Copilot, CLI) et les équipes qui veulent garder leur code en local ; côté consommateur, `pnpm add -D code-dev-intel.ts` puis `pnpm exec code-dev-intel ensure --workspaceRoot=. --port=4545` démarre le service seulement s'il ne tourne pas déjà.

Outils exposés : `findDefinitions`, `findReferences`, `findImplementations`, `findSymbol`, `findCallers` / `findCallees`, `getSymbolContent`, `getFileOutline`, `dependencyGraph`, `impactedFiles` (résolus par le type-checker), plus `searchStruct` (ast-grep), `searchText` (ripgrep) et `findDuplicates`.

Protocoles : MCP sur stdio, MCP JSON-RPC sur `POST /mcp`, endpoints HTTP `/tools/*`, santé et découverte sur `/health` et `/tools/describe`.

Dépôt personnel créé le 2026-03-20, dernier push le 2026-09-06, licence AGPL-3.0 (copyleft, contraignante pour un usage commercial), 0 étoile, 0 fork : un outil d'auteur, pas encore une communauté.

Langages : donnée non relevée ; à vérifier dans <https://github.com/jordanvalnet/code-dev-intel.ts>. Ce qu'on sait : tout est TypeScript en ESM (`"type": "module"`, `tsconfig.json`, `vitest.config.ts`, `eslint.config.mjs`).

Comment le code est-il organisé ?

> Tout le code vit sous `services/` ; le reste de la racine sert à le tester, le livrer et le mesurer.

- └ `.github/` workflows GitHub Actions
- └ `__tests__/` tests unitaires vitest
- └ `docker/` `docker-compose.yml` et profils
- └ `docs/` documentation, dont `docs/ai`
- └ `perf/` benchmarks de performance
- └ `schemas/` schémas livrés avec le paquet
- └ `scripts/` bootstrap, smoke tests, benchmark
- └ `security/` ressources sécurité
- └ `services/` serveur MCP, indexeur, gouvernance, sécurité
- └ `.dockerignore` exclusions du build Docker
- └ `.gitignore` fichiers ignorés par git
- └ `.npmignore` exclusions du paquet npm
- └ `CHANGELOG.md` journal des versions
- └ `CONTRIBUTING.md` guide de contribution
- └ `LICENSE` licence AGPL-3.0-only
- └ `README.md` présentation et documentation principale

... et 6 autres entrées à la racine du dépôt

`services/` porte tout le code : le serveur MCP (`services/code-intel-mcp/`), l'indexeur (`services/indexer/`), la gouvernance (`services/governance/`) et la sécurité (`services/security/`).

`__tests__/unit/` : 4 fichiers de tests vitest, un sous-dossier par service (code-intel-mcp, governance, indexer, security).

`scripts/` (bootstrap, smoke tests, benchmark), `perf/` (benchmarks de performance), `schemas/` (schémas livrés avec le paquet) et `security/` (règles du scan) sont l'outillage autour du produit.

`docker/` contient `docker-compose.yml` avec les profils `core` , `search-optional` et `zoekt-optional` (`pnpm docker:core:up`); `docs/` regroupe `docs/ai/` (backlog et mémoire des agents), `docs/benchmarks/` et `docs/design/` .

`tsconfig.build.json` compile vers `dist/` ; le paquet publié ne contient que `dist/` , `schemas/` , `CHANGELOG.md` , `LICENSE` et `README.md` , et `.github/` abrite les quatre workflows Actions.

Par où commencer à lire ?

> `server.ts` d'abord, `contracts.ts` ensuite : tout le reste des services s'y branche.

1. `services/code-intel-mcp/src/server.ts` : le serveur MCP, `main` et `bin` (`code-dev-intel`) du paquet une fois compilé dans `dist/` , lancé par `pnpm mcp:start` , auto-testé par `--self-test` en CI.
2. `services/code-intel-mcp/src/contracts.ts` : les contrats des outils exposés, export public `./contracts` du paquet.
3. `services/code-intel-mcp/src/mcp-handler.ts` et `health-handler.ts` : le texte d'instructions envoyé au client MCP et la description des endpoints `/tools/*` (le routage « symbole → outil, texte → `searchText` »).
4. `services/indexer/src/indexer-runner.ts` : l'indexeur, `pnpm indexer:impacted` , piloté par `scripts/indexer-smoke.mjs` en modes git-diff et impacted.
5. `services/security/opengrep-runner.ts` (`pnpm security:scan` , job security de la CI) puis `scripts/bootstrap.sh` , qui vérifie `docs/ai/*` et crée les dossiers de travail.

Garde le README ouvert à « Typical agent workflow » (`getFileOutline` → `getSymbolContent` → `findReferences` → `dependencyGraph` → `searchStruct` → `searchText`) : c'est la carte des outils que `server.ts` branche.

05

Comment builder, tester et lancer ?

```
> pnpm install --frozen-lockfile puis pnpm ci:quality sous Node 24 : la CI en local, avant la première PR.
```

Les trois commandes du poste de travail, dans l'ordre :

```
pnpm install --frozen-lockfile
pnpm ci:quality
pnpm mcp:self-test
```

`pnpm` seulement (npm et yarn interdits) ; `pnpm ci:quality` enchaîne `pnpm lint`, `pnpm type-check` et `pnpm test` (vitest, `__tests__/unit`), les trois gates obligatoires de toute PR ; `pnpm mcp:start` lance le serveur depuis le TypeScript via `node --experimental-strip-types`, `pnpm build` compile vers `dist/`.

Node : `engines` annonce node `>=18`, mais `vitest 5` et `--experimental-strip-types` exigent node 22+ ; la CI `.github/workflows/ci.yml` tourne sur push et pull_request en ubuntu, windows et macos avec node 20, 22 et 24 (lint, type-check, tests, puis release-smoke, indexer-smoke et scan OpenGrep), les autres workflows sont `perf-budget.yml`, `pr-memory-reference.yml` et `security.yml` (déclencheurs non relevés ; à vérifier dans <https://github.com/jordanvalnet/code-dev-intel.ts/tree/main/.github/workflows>).

Comment le projet vit-il ?

> Un projet d'une seule personne qui avance par rafales : la dernière, 11 commits, s'est terminée le 2026-09-06.



12 commits sur les douze semaines relevées, tous de jordanvalnet : bus factor 1, une seule personne porte le projet.

Un rythme par rafales : 11 commits la semaine 2026-W36, 0 les semaines précédentes du relevé, dernier commit le 2026-09-06.

Cette rafale correspond à la PR #3 « ci: three-OS quality matrix, multi-OS release smoke, per-OS perf budget (T-021) », fusionnée le 2026-09-06 : la CI est passée d'un runner Linux à ubuntu, windows et macos, ce qui a fait remonter un bug de canonicalisation de chemin dans `collectWorkspaceFiles`¹.

Releases : une seule taguée sur GitHub, v0.1.6 du 2026-03-26, alors que `CHANGELOG.md` et `package.json` documentent des versions ultérieures et une section Unreleased ; la version publiée sur npm est à vérifier dans <https://github.com/jordanvalnet/code-dev-intel.ts/blob/main/CHANGELOG.md>.

Repères pour agents IA à la racine (AGENTS.md, CLAUDE.md, .cursorrules, .mcp.json...) : 0 sur 7, alors que le projet impose un protocole de mémoire partagée dans `docs/ai/` ; pas de SECURITY.md ni de CODEOWNERS non plus.

Jalons et CVE : données non relevées le 2026-09-10 ; à vérifier dans <https://github.com/jordanvalnet/code-dev-intel.ts/milestones> et <https://github.com/jordanvalnet/code-dev-intel.ts/security>.

Quelle première contribution ?

> Pas de good first issue : ta première PR sera petite, sur une branche `task/T-00X`, et elle passera par la mémoire partagée.

Aucune issue « good first issue », aucun label : 1 issue ouverte, 0 fermée sur 30 jours, 0 PR ouverte.

L'unique issue, #1 « Anti-Amnesia & Workflow Hardening for code-dev-intel.ts », 0 commentaire, est le chantier ouvert : lis-la avant de choisir ta tâche².

Le process est strict : suivre `docs/ai/04-executable-task-backlog.md` dans l'ordre, une branche `task/T-00X-short-name` par tâche et par PR, chaque événement ajouté à `docs/ai/memory/AGENT_MEMORY.md`, ce que `pnpm pr:check-memory-reference` (`services/governance/pr-memory-reference-check.ts`) contrôle.

CHANTIER	POURQUOI MAINTENANT
Ajouter <code>SECURITY.md</code>	aucun canal déclaré pour signaler une faille, risque noté moyen
Ajouter <code>AGENTS.md</code> à la racine, pointant sur <code>docs/ai/</code>	0 repère IA sur 7 alors que le projet est fait pour des agents
Taguer la version du manifeste	une seule release GitHub, v0.1.6, loin derrière le changelog

Définition de fini : critères du backlog remplis, preuves (commandes et sorties), mémoire partagée à jour, prochaine action de handoff écrite, `pnpm lint`, `pnpm type-check` et `pnpm test` verts.

\$ Vos 3 premières actions

1. Aligner ton poste sur la CI, sous Node 24 : `git clone https://github.com/jordanvalnet/code-dev-intel.ts && cd code-dev-intel.ts && pnpm install --frozen-lockfile && pnpm ci:quality` ; vert = tu as le même résultat que `.github/workflows/ci.yml` .
2. Faire tourner le serveur et le lire : `pnpm mcp:self-test` , puis `pnpm mcp:start:logs` avec le README à « Typical agent workflow », en suivant `services/code-intel-mcp/src/server.ts` puis `services/code-intel-mcp/src/contracts.ts` .
3. Ouvrir ta première PR dans le process : lire l'issue #1 et `docs/ai/04-executable-task-backlog.md` , créer `task/T-0XX-security-md` , ajouter `SECURITY.md` , noter l'événement dans `docs/ai/memory/AGENT_MEMORY.md` , pousser² .

RÉFÉRENCES PUBLIQUES

Sources

1. <https://github.com/jordanvalnet/code-dev-intel.ts/pull/3>
2. <https://github.com/jordanvalnet/code-dev-intel.ts/issues/1>